Práctica 3

Nolan Tannion Rodríguez

Computación Avanzada
Universidad Autónoma de Madrid

Resumen

Cálculo de la trayectoria de una pelota de golf ante un golpe *slice* y un *hook* usando el método de Euler, apreciando una curvatura. Se compara usando: un integrador Runge-Kutta 4, velocidad angular sobre varios ejes y distintos modelos de rozamiento. Se observa que el cambio de integrador no afecta al resultado, sin embargo la rotación y la fricción varían las trayectorias.

1. Introducción teórica

1.1. Ecuaciones de movimiento

El movimiento de la bola de golf se modela como el de un proyectil sometido a tres fuerzas principales: gravedad, arrastre aerodinámico y efecto Magnus. La ecuación general del movimiento es

$$m\frac{d\vec{v}}{dt} = \vec{F}_g + \vec{F}_d + \vec{F}_M.$$

La gravedad actúa en la dirección vertical,

$$\vec{F}_g = -mg\,\hat{y}.$$

El arrastre aerodinámico se toma proporcional al cuadrado de la velocidad y opuesto al movimiento,

$$\vec{F}_d = -\frac{1}{2}\,\rho\, C_d\, A\,|\vec{v}|\,\vec{v},$$

donde ρ es la densidad del aire, A la sección transversal de la bola y C_d un coeficiente efectivo que puede depender de la velocidad.

Se considerará la densidad del aire como constante pues el rango de alturas en el que se trabaja, usar un modelo isotermo aumentaría el número de cálculos realizados y supondría una diferencia de $\approx 0,3\,\%$ en el punto de máxima altura. Al usar un intervalo de integración $h = 0,001$ este error sería del mismo orden que el error causado por la integración, por lo que no mejoraría nuestra precisión incluir la variación.

El efecto Magnus aparece cuando la bola rota con velocidad angular $\vec{\omega}$ y se modela como

$$\vec{F}_M = S_0 \cdot \vec{\omega} \times \vec{v},$$

siendo S un parámetro que depende del objeto. Esta fuerza es perpendicular a $\vec{v}$ y produce desviaciones laterales responsables del *hook* y el *slice*.

Se analiza la energía mecánica:

$$E = \frac{1}{2}mv^2 + mgy,$$

lo que permite cuantificar el efecto disipativo del arrastre y comparar la precisión de los distintos integradores.

1.2. Teoría computacional

El sistema se expresa como un conjunto de ecuaciones diferenciales para las variables

$$(x, v_x, y, v_y, z, v_z),$$

Se reducen las ecuaciones a primer órden duplicando el número de ecuaciones y de condiciones iniciales necesarias. Se expresan de ecuaciones de la siguiente manera:

$$\frac{dx}{dt} = v_x; \quad \frac{dv_x}{dt} = F_x(\vec{v}); \quad \cdots$$

La integración temporal se realiza con paso fijo h. Dado un sistema dinámico con estructura:

$$\frac{d\vec{y}}{dt} = f(t, \vec{y});$$

Principalmente se discretizará usando el método de Euler, que calcula el paso posterior según la expresión:

$$t_{n+1} = t_n + h \quad \vec{y}_{n+1} = \vec{y}_n + h\,\vec{f}(\vec{y}_n),$$

Se incluye también el método de Runge-Kutta de cuarto orden para mejorar la precisión combinando evaluaciones intermedias con distintos pesos:

$$\vec{y}_{n+1} = \vec{y}_n + \frac{h}{6}\left(\vec{k}_1 + 2\vec{k}_2 + 2\vec{k}_3 + \vec{k}_4\right),$$

Los incrementos intermedios vienen dados por:

$$\vec{k}_1 = \vec{f}(t_n, \vec{y}_n),$$

$$\vec{k}_2 = \vec{f}\left(t_n + \frac{h}{2}, \vec{y}_n + \frac{h}{2}\vec{k}_1\right),$$

$$\vec{k}_3 = \vec{f}\left(t_n + \frac{h}{2}, \vec{y}_n + \frac{h}{2}\vec{k}_2\right),$$

$$\vec{k}_4 = \vec{f}(t_n + h, \vec{y}_n + h\vec{k}_3).$$

La simulación se detiene cuando la coordenada vertical se anula, identificando el impacto con el suelo. A partir de la solución numérica se obtiene la trayectoria tridimensional.

2. Implementación

Se procederá a incluir y comentar las principales secciones del código

2.1. Estructura general

2.1.1. Definición de constantes físicas

Se fijan los parámetros físicos del sistema: velocidad inicial, masa y radio de la bola, gravedad, densidad del aire, parámetro asociado al efecto Magnus y ángulo de salida. Estas constantes se mantienen fijas durante toda la simulación.

El ángulo de salida se ha fijado a 15º en base a la Referencia [1], siendo un ángulo intermedio de golpeo. La masa y radio de la bola se obtienen del reglamento oficial que se encuentra en [2].

```
v0 = 70              # Velocidad inicial (m/s)
m = 0.04593          # Masa de la bola (kg)
r_bola = 0.0213      # Radio (m)
A = np.pi*r_bola**2
g = 9.81
S0wm = 0.25          # Parametro del efecto
    Magnus (s^-1)
ang = np.deg2rad(15) # Direccion del golpeo de
     la bola
```

2.1.2. Integradores

Se implementan ambos como funciones que toman los inputs necesarios comentados en bloque al inicio: intervalo de integración, paso de integración, condiciones iniciales. etc. Las funciones devuelven un array con las soluciones para cada variable, además de un array con los tiempos. Se añade una condición que detiene la integración antes del tiempo máximo si la pelota golpea el suelo.

```python
def euler(sistema,t0,tf,h,estado0, signo, rhof):
    """
    INPUTS:
    - sistema: funcion que define las derivadas de cada variable
    - h: longitud del paso para integrar
    - t0: tiempo inicial
    - tf: tiempo maximo de simulacion
    - estado0: estado inicial del sistema
    - signo: signo que describe el sentido de la rotacion
    - rho: funcion que calcula la densidad del aire a la altura dada

    RETURN:
    - vars: np.array de dimensiones (N,6)
    - t: array de longitud N

    """
    N = int(abs(tf-t0) / h)
    vars = np.zeros((N, 6))
    vars[0] = estado0
    t = [t0]

    for i in range(N - 1):
        rho = rhof(vars[i, 2])  # densidad en funcion de y
        vars[i+1] = vars[i] + sistema(vars[i], rho, signo) * h
        t.append(t[i] + h)

        # impacto con el suelo e interpolacion
        if vars[i+1, 2] < 0:
            interp = (vars[i+1] + vars[i])/2
            vars[i+1] = interp
            t[i+1] = t[i] + h/2
            return t,vars[:i+2]

    return t, vars
```

Listing 1: Integrador de Euler

```python
def rk4(sistema, t0, tf, h, estado0, signo, rhof):
    '''
    INPUTS:
    sistema : funcion que define la dinamica del sistema
    t0      : tiempo inicial
    tf      : tiempo final
    h       : paso de integracion
    estado0 : array con el estado inicial [x, vx, y, vy, z, vz]

    ADICIONAL (para los casos de proyectil)
    signo   : signo del giro (+1 slice, -1 hook)
    rhof    : metodo para calcular la densidad del aire en funcion de la altura

    RETURNS:
    t    : array de tiempos
    vars : array con la evolucion del sistema,
           vars[i] = [x, vx, y, vy, z, vz] en t[i]
    '''

    N = int(abs((tf - t0)) / h)

    vars = np.zeros((N, 6))
    t = np.zeros(N)

    vars[0] = estado0
    t[0] = t0

    for i in range(N - 1):
        y = vars[i]

        rho1 = rhof(y[2])
        k1 = sistema(y, rho1, signo)

        rho2 = rhof(y[2] + 0.5*h*k1[2])
        k2 = sistema(y + 0.5*h*k1, rho2, signo)

        rho3 = rhof(y[2] + 0.5*h*k2[2])
        k3 = sistema(y + 0.5*h*k2, rho3, signo)

        rho4 = rhof(y[2] + h*k3[2])
        k4 = sistema(y + h*k3, rho4, signo)

        vars[i+1] = y + (h/6)*(k1 + 2*k2 + 2*k3 + k4)
        t[i+1] = t[i] + h

        # impacto con el suelo (interpolacion lineal)
        if vars[i+1, 2] <= 0:
            y1 = vars[i, 2]
            y2 = vars[i+1, 2]

            # Parametro para la interpolacion
            interp =  (y2 - y1)/2

            vars[i+1] = vars[i] + interp *h/2
            t[i+1] = t[i] + h/2

            return t[:i+2], vars[:i+2]

    return t, vars
```

Listing 2: Integrador de Runge-Kutta 4

Pese a usar la densidad del aire constante para este problema, la función de la dinámica y de integración incluyen un input de una función de cálculo de la densidad con finalidad de generalidad y reutilización del código.

2.2. Modelos de arrastre aerodinámico

Para estudiar la influencia del rozamiento con el aire, se han implementado distintos modelos de arrastre con diferente nivel de complejidad. Esto permite comparar la sensibilidad de la trayectoria y de la energía a la forma concreta en la que se introduce la disipación.

2.2.1. Arrastre longitudinal simplificado

Inicialmente se emplea un modelo en el que el arrastre solo actúa sobre la componente x, mientras que se desprecia en las demás direcciones.

```python
def arrastrex(var):
    '''
    Calcula el arrastre en la direccion x
    INPUT
    -var array que contiene todas las variables

    RETURNS
    - C: coeficiente de arrastre
    '''
    C = [1,0,0]
    if abs(var[1]) > 14:
        C[0] = 14/abs(var[1])

    return C
```

2.2.2. Arrastre completo

Para una descripción más realista, se define un coeficiente de arrastre independiente para cada componente de la velocidad. En este caso, el rozamiento actúa de forma distinta en cada eje y depende del régimen de velocidad en cada dirección.

```python
def arrastre_completo(var):
    '''
    Calcula el arrastre en todas las direcciones
    INPUT
    -var array con todas las variables

    RETURNS
    - C: coeficiente de arrastre
    '''

    C = [1,1,1]
    if abs(var[1]) > 14:
        C[0] = 14/abs(var[1])

    if abs(var[3]) > 14:
        C[1] = 14/abs(var[3])

    if abs(var[5]) > 14:
        C[2] = 14/abs(var[5])

    return C
```

2.3. Ecuaciones del movimiento

Se define una función para la dinámica del sistema. Los inputs están especificados en el docstring de la función, siendo especialmente importante vars que contiene las variables, el return de la función es un array con la misma estructura que vars conteniendo la derivada de cada variable.

Se define un array que asocie +1 al Hook y -1 al Slice. Asignados de esa manera por los signos elegidos en el parámetro de rotación S_0.

```python
def sistema(vars, rho, sign):
    '''
    Funcion que define la dinamica del sistema.

    INPUTS

    - vars: array con estructura [x,vx,y,vy,z,vz]
    - rho: funcion para calcular la densidad en funcion de la altura
    - sign: array de valor  +1 o -1 que indica el sentido de rotacion de la pelota

    RETURNS
    - dvars: array que contiene la derivada de cada variable de vars
    '''

    # Creamos el array que almacena las derivadas de cada variable
    dvars = np.zeros_like(vars)

    # Modulo de la velocidad
    modv = np.sqrt(vars[1]**2 + vars[3]**2 + vars[5]**2)

    # Coeficiente de arrastre como funcion de v
    C = arrastrex(v=vars)

    # Calculamos la derivada de cada variable de posicion
    dvars[0] = vars[1]
    dvars[2] = vars[3]
    dvars[4] = vars[5]

    # No incluimos el arrastre en z e y
    arrastre = -(1/2)*rho*A/m *modv
    dvars[1] = arrastre*vars[1]*C[0]
    dvars[3] = -g + arrastre*vars[3]*C[1]
    dvars[5] = S0wm*vars[1] * sign + arrastre*vars[5]*C[2]

    return dvars
```

En función de las ampliaciones llevadas a cabo se modifica ligeramente la función de la dinámica.

2.3.1. Considerar todo el arrastre

Se cambia:

```python
# Coeficiente de arrastre como funcion de v
C = arrastrex(v=vars)
```

Por:

```python
# Coeficiente de arrastre como funcion de v
C = arrastre_completo(v=vars)
```

2.3.2. Rotación sobre varios ejes

Se crea un array con la dirección de la rotación, se mantiene el módulo siendo únicamente cambio del eje de rotación:

```python
rotacion = -[S0wm* np.cos(45), S0wm*np.sin(45), 0]
```

Se calcula la Fuerza Magnus con el producto vectorial y se incluye en cada eje:

```python
Fmag = np.cross(rotacion,vel) *sign
dvars[1] = arrastre*vars[1]*C[0] + Fmag[0]
dvars[3] = -g + arrastre*vars[3]*C[1] + Fmag[1]
dvars[5] = arrastre*vars[5]*C[2] + Fmag[2]
```

2.4. Integración numérica

El sistema se resuelve mediante integración temporal con paso fijo usando el método de Euler o Runge-Kutta de cuarto orden. La integración se detiene cuando la coordenada vertical se hace negativa o cero, identificando el impacto con el suelo. Se definen también las condiciones de integración.

```python
# Integracion
t0 = 0
h = 1e-3
tmax = 100

t, sol = euler(sistema, t0, tmax, h, estado, sign, rho_const)
trk, solrk = rk4(sistema, t0, tmax, h, estado, sign, rho_const)
```

Las variables trk, solrk también se utilizan para los resultados ante las distintas variaciones introducidas:

```python
t, sol = euler(sistema_x,t0,tmax, h, estado, sign, rho_const)
trk, solrk = euler(sistema_completo,t0,tmax, h, estado, sign, rho_const)
```

Aquí varía la función de la dinámica, con una incluyendo el arrastre en x y la otra en todas las direcciones. De esta manera se calculan y almacenan los dos resultados para comparar la influencia de las variaciones introducidas.

2.5. Desplazamiento máximo

Se muestra en pantalla el valor absoluto del desplazamiento máximo para cada eje:

```
print(f'Desplazamiento maximo en |x|: {max(abs(sol[:,0]))}')
print(f'Desplazamiento maximo en |y|: {max(abs(sol[:,2]))}')
print(f'Desplazamiento maximo en |z|: {max(abs(sol[:,4]))} \n')
```

2.6. Guardado de la gráfica

Para almacenar los gráficos usamos:

```
.savefig('Nombre.png', dpi = 500)
```

Antes del punto se debe incluir plt o el nombre que se le haya asignado a la figura que se desee guardar. Se almacenará según el nombre indicado, dpi es un parámetro para controlar la calidad de guardado de la imagen, se fija en 500 para obtener una imagen de mejor resolución.

3. Resultados

3.1. Trayectoria de la pelota, cálculo inicial

Representando la trayectoria en 3 dimensiones obtenemos la Figura 1. Además en la Figura 7 podemos ver que el golpe Hook se curva en la dirección z positiva, mientras que el Slice en la opuesta. Esto coincide con lo esperado teóricamente por los signos de la velocidad angular y el producto vectorial.

El punto de aterrizaje de la bola ha sido:

$$\begin{pmatrix} 148{,}521 & 0 & \pm 82{,}589 \end{pmatrix}$$

Donde + corresponde al Hook y - al Slice.

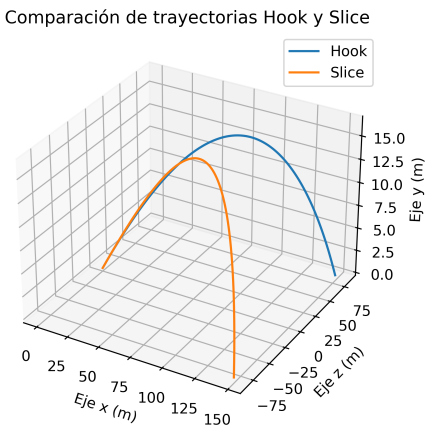

Figura 1: Trayectoria tridimensional de la pelota para cada tipo de golpe. Sólo se tiene en cuenta el rozamiento del aire en la dirección x.

3.2. Arraste longitudinal o completo

Las figuras 8 y 2 representan las variaciones de la trayectoria según el modelo de rozamiento que se haya utilizado. Esta sección es donde se han visto los cambios más drásticos de la trayectoria. En la representación de la energía podemos ver que la energía decae más rápidamente cuando se considera el arrastre completo, lo que es coherente con el resultado esperado. Las diferencias entre los puntos de aterrizaje y las energías:

- Hook: 49,199m
- Slice: 49,199m
- Energía: 14,882J

El punto de aterrizaje de la bola usando el arrastre completo es:

$$\begin{pmatrix} 119{,}059 & 0 & \pm 20{,}076 \end{pmatrix}$$

Donde + corresponde al Hook y - al Slice

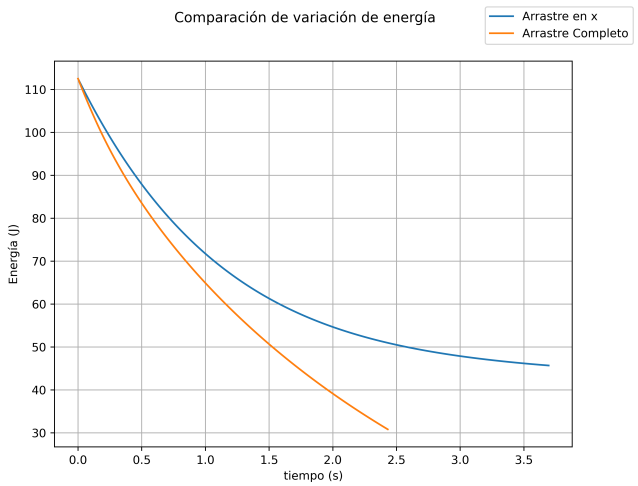

Figura 2: Representación de la suma de la energía cinética y potencial en función del tiempo. Se comparan los modelos de rozamiento utilizados.

En las siguientes secciones se continúa utilizando el arrastre en todos los ejes en vez del longitudinal.

3.3. Efecto del integrador

Observando las Figuras 3 y 6 vemos que tanto las trayectorias ni las energías tienen una variación notable. Esta comparación justifica el uso del método de Euler, pues es menos costoso computacionalmente. Es un resultado esperable pues la trayectoria de la bola es bastante suave y no tiene fuertes oscilaciones.

El punto de aterrizaje usando el método de Runge-Kutta es:

$$\begin{pmatrix} 118{,}997 & 0 & \pm 20{,}059 \end{pmatrix}$$

La distancia entre los puntos de aterrizaje en función del método ha sido:

- Hook: 0,0641m
- Slice: 0,0641m
- Energía: 0,028J

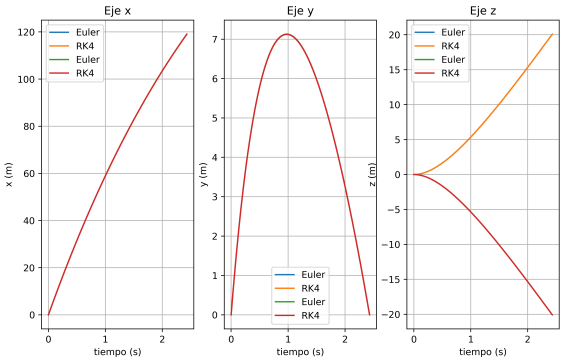

Figura 3: Comparación de las trayectorias de la pelota en cada dirección en función del tiempo. Calculadas con el método del Euler y Runge-Kutta de 4º órden.

3.4. Efecto de rotar sobre otro eje

Estudiando el caso de un Hook o Slice imperfectos cuya rotación no sea exactamente la deseada podemos ver que si hay una variación del punto de caída que podría marcar la diferencia en una competición real. Podemos ver estos resultados en las Figuras 4 y 5. El punto de aterrizaje para el giro con un eje distinto es:

$$\begin{pmatrix} 120{,}873 & 0 & \pm 18{,}055 \end{pmatrix}$$

La distancia entre los puntos de aterrizaje esta vez ha sido:

- Hook: 1,474m
- Slice: 1,474m

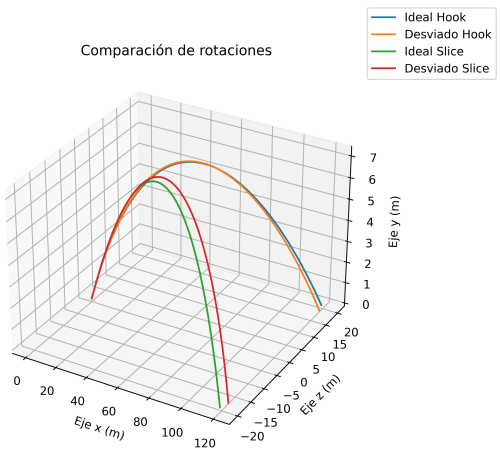

Figura 4: Comparación de las trayectorias que toman las bolas en función de un cambio en la dirección de rotación de la pelota además del tipo de golpe.

4. Conclusiones

El modelizado de la trayectoria tridimensional de una pelota de golf bajo la acción de la gravedad, el arrastre aerodinámico y el efecto Magnus ha reproduciendo correctamente la curvatura lateral asociada a golpes hook y slice. Identificándose el efecto Magnus como el principal responsable de la desviación en el eje z.

La comparación entre los métodos de Euler y Runge–Kutta de cuarto orden muestra diferencias despreciables para el paso temporal empleado, lo que justifica el uso del integrador de Euler por su menor coste computacional. Asimismo, se observa que la forma en la que se modeliza el arrastre aerodinámico tiene una influencia significativa tanto en la trayectoria como en la disipación de energía mecánica.

Finalmente, variaciones en la dirección del eje de rotación producen cambios apreciables en el punto de caída, mostrando la sensibilidad del sistema a las condiciones iniciales y los efectos aerodinámicos considerados.

Referencias

[1] GlobalGolf. Pro tip: Launch monitor guide – know your numbers (and what they mean), December 2020. Consultar en: ://www.globalgolf.com/articles/pro-tip-61/ . Actualizado: 15 Mar 2023.

[2] Real Federación Española de Golf. Guía a las reglas sobre palos y bolas. PDF, 2010. Edición 3.1, Comité Técnico de Reglas.

Gráficas

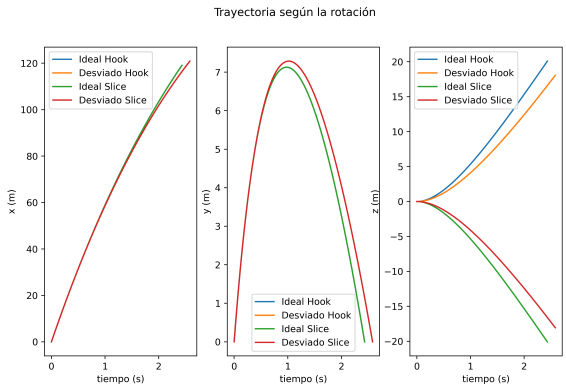

Figura 5: Representación de la evolución temporal de cada coordenada en función de del tipo de golpe y si la rotación estaba desviada.

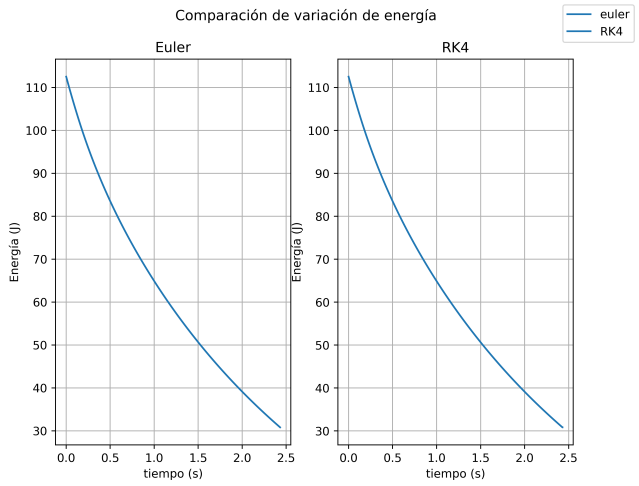

Figura 6: Comparación de la suma de energía cinética y potencial a lo largo del tiempo según el integrador utilizado.

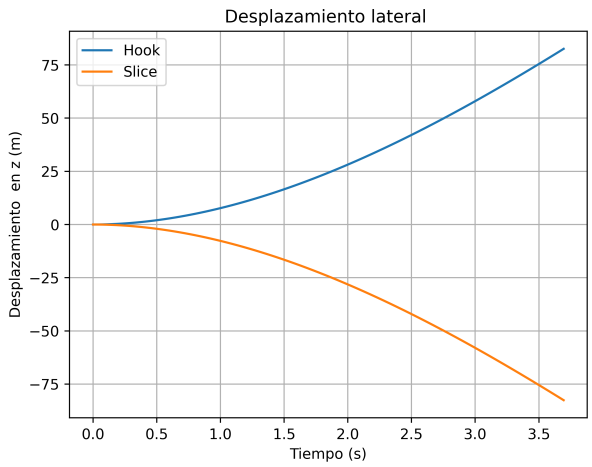

Figura 7: Posición en el eje z en función del tiempo y del tipo de golpe. De nuevo solo teniendo en cuenta el rozamiento en x.

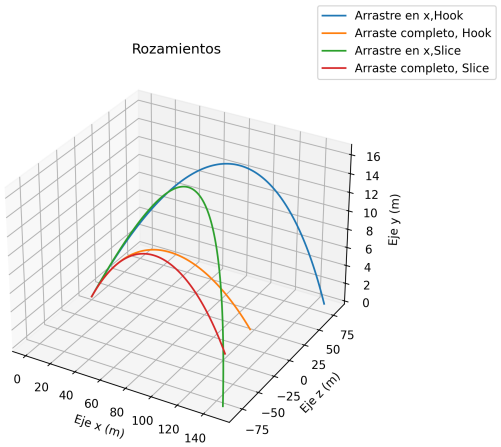

Figura 8: Representación tridimensional de las trayectorias de la bola en función del tipo de golpe y el modelo de arrastre utilizado.